BASIC overview;
* Biggest bottleneck is the time required to perform unions + joins in non-cmath DBs.
* The "matching" process of comparing labels is accelerated by replacing the URI string with shorter, numerical terms.

Complex Query Types:
- Multi-entity
- Type
- Temporal
- Ordinal
- Aggregation.

* So what:
  - Inference is what makes KGs interesting
  - How graph is stored makes a difference to retrieval speed.

A Characterization of Knowledge Graph Workloads

Ben Landrum

December 23, 2022

Abstract

Knowledge graphs (KGs) are a means of representing relational information in the form of a graph, where nodes represent entities and directed edges represent relationships between them. We give background on their history and the RDF and SPARQL standards used to represent and query them. We then present an overview of the application of knowledge graphs to the domain of question answering, and introduce some common types of queries used in this context and the bottlenecks in their execution.

1 Introduction

A knowledge graph (KG) is a graph representation of relational information, where nodes represent nouns and directed edges represent relationships between them. This is typically stored in the form of (*head*, *label*, *tail*) triples, denoted (h, ℓ, t), and sometimes referred to as *subject-predicate-object* triples[15], where *head* has a relationship such as "educated at" with *tail*, in this case representing that *head* (a person) was educated at *tail* (an institution). The label is a string representing the relationship between the head and tail. The graph is then constructed by adding all of the triples as edges[14].

Formally, the term knowledge graph includes any graph representing discrete relationships between entities [14]. However, virtually all research explicitly studying topics related to 'knowledge graphs' is concerned chiefly with the case of knowledge graphs of general-purpose, publicly available information about real-world entities, as opposed to those representing connections in a social network or another specialty domain. The first large-scale example of this type of knowledge graph was the Freebase project, a large-scale open source knowledge graph started in 2007 that was meant to structure information from the internet in a form more machine readable than raw text but more flexible than a traditional relational database[7, 8]. In 2010, Google acquired Metaweb, the company behind Freebase, and in 2012 popularized the term 'knowledge graph' with the introduction of the Google Knowledge Graph, an extension of Freebase used for question answering within Google search [20]. The Google Knowledge Graph is a large-scale knowledge graph of real-world entities, with over 500 million entities and 3.5 billion relationships at the time of its announcement[20].

Other large-scale knowledge graphs, such as YAGO[18], DBpedia[16], and Wikidata[21], have been created by the open source community, and are used for a variety of purposes, including question answering, entity linking, and entity resolution. These knowledge graphs are

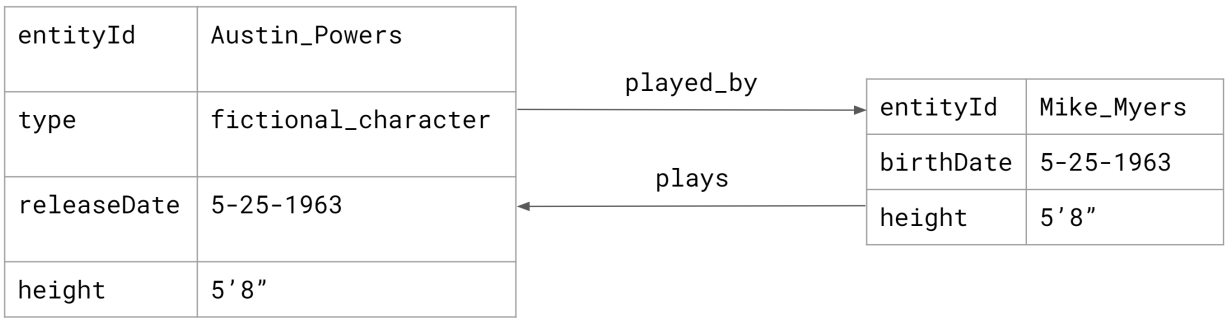


Figure 1: A reciprocal pair of relations in a hypothetical knowledge graph, representing the RDF triples `(Austin_Powers, played_by, Mike_Myers)` and `(Mike_Myers, plays, Austin_Powers)`. Note that not all information is necessarily a relation, as entities can have non-relational properties such as height.

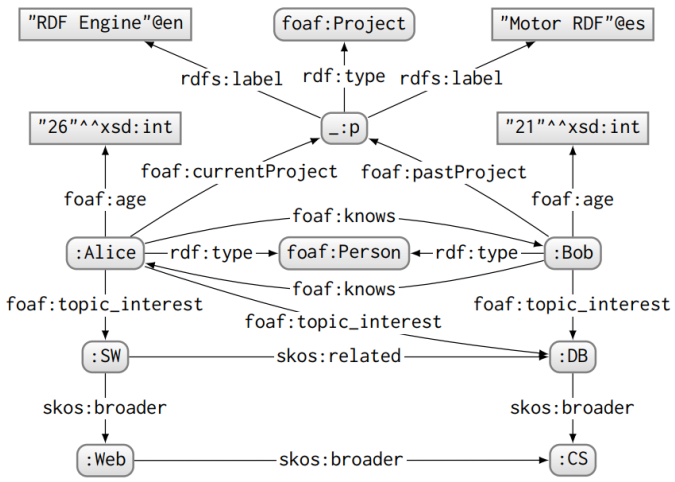
built with varying amounts of machine interpretation of Wikipedia articles, crowd-sourced data, and refinements of other open source knowledge base projects[18].

2 RDF and SPARQL

The (*head*, *label*, *tail*) notation is derived from the Resource Description Framework (RDF) specification, which is the most common format for storing and operating on knowledge graphs[14]. While talking specifically about RDF triples, we will use the more standard notation from the literature specific to RDF of (*subject*, *predicate*, *object*) (or (s, p, o)) in lieu of (*head*, *label*, *tail*) to be more consistent with related publications. RDF is a standard for representing relational information in the form of triples, and was developed by the World Wide Web Consortium (W3C) as part of a broader initiative known as the Semantic Web which aims to make the internet more easily machine readable[6, 15].

All three values in an RDF triple may be URIs, which are globally unique identifiers for resources. The object can be a non-URI literal value, which is how, for example, height would be represented in the graph depicted in figure 1. It's also valid for the subject or predicate of a triple to be the so-called "blank node", which can represent something specific to the context of the graph or an entity that doesn't have a URI[15]. URIs are generally a string, often a URL, that uniquely identifies a resource. For example, the URI for the entity representing the character Austin Powers in DBpedia is a link to a page describing the entity: "https://dbpedia.org/page/Austin_Powers_(character)"[16]. Implementationally, it can save memory and accelerate comparisons to use a numeric identifier instead of a string for the URI and then use a lookup table to map the numeric identifier to the string URI when returning a result. This approach is ideal for workloads involving many complex queries because the cost of the lookup when returning the result is offset by the accumulated benefit from internal comparisons.[2]

The primary bottleneck in the querying of a knowledge graph built on an RDF database is the time required to perform large joins and unions[2, 11]. The design of RDF storage



Subject	Predicate	Object
:Alice	rdf:type	foaf:Person
:Alice	foaf:age	"26"^^xsd:int
:Alice	foaf:topic_interest	:DB
:Alice	foaf:topic_interest	:SW
:Alice	foaf:knows	:Bob
:Alice	foaf:currentProject	_:p
:Bob	rdf:type	foaf:Person
:Bob	foaf:age	"21"^^xsd:int
:Bob	foaf:topic_interest	:DB
:Bob	foaf:knows	:Alice
:Bob	foaf:pastProject	_:p
_:p	rdf:type	foaf:Project
_:p	rdfs:label	"RDF Engine"@en
_:p	rdfs:label	"Motor RDF"@es
:SW	skos:broader	:Web
:SW	skos:related	:DB
:Web	skos:broader	:CS
:DB	skos:broader	:CS

Figure 2: A triple table and the associated knowledge graph. Taken from [2].

systems thus prioritizes physical storage and indexing which minimizes the overhead and number of these operations required.

RDFS RDF does not natively restrict the types of predicates that can be associated with a given subject or make guarantees about the datatypes of objects, with the aim of maximizing flexibility and facilitating the addition of new types of data with minimal overhead[15]. This has several drawbacks for efficient storage and querying of RDF databases[11, 2]. An extension of RDF called *RDF Schema* (RDFS) was developed to address these issues by introducing a way to impose schema on RDF data. RDFS is a vocabulary of RDF triples that can be used to add restrictions to the RDF graph, such as specifying that a given predicate can only be associated with a given subject type, or that a given object must be a certain datatype[12]. Most prominently, schema are used to define classes of objects like `Person` and `Movie`, and describe the properties which can be expected to be associated with each class. For example, Schema.org, a widespread schema used by projects such as YAGO[18] specifies that a `Person` has a `givenName` property, a `Movie` has a `name` property, and both have an *expected type* of `Text`[19].

2.1 RDF Storage

The manner in which an RDF database is stored has significant impacts on the efficiency with which it can be queried, updated, or compressed, and thus on the performance of applications built on top of it. What follows is an overview of different categories of RDF storage methods, and the advantages and disadvantages of each, following loosely the categorizations used in [11] and [2].

2.1.1 Triple Tables

A *triple table* is the intuitive, naive approach to storing an RDF graph, where each triple is a row in a table with a three column schema [11, 2]. This approach is adequate for the simplest kind of query, *statement-based* queries, which provide a single pattern that the results must match, such as specifying a subject and a predicate and returning all objects that match. However, it is not well suited to more complex operations, which will require many joins over the full table[11]. An example of a triple table is shown in figure 2.

2.1.2 Property Tables

A *property table* represents predicates as columns and subjects as rows in a table where a row's value in a given column corresponds to the object(s) appearing with the corresponding subject/predicate pair. This table is indexed by subject to allow for efficient lookups of all properties of a given entity. As a result, when a query calls for finding multiple properties of a given subject (such as when filtering with two criteria involving different properties), no joins are required[11]. This approach has the considerable drawback of creating many `NULL` entries, as in practice knowledge graphs are relatively sparse and many subjects will not have a value for a given predicate. The number of these `NULL` entries is often reduced in schema-less RDF databases through clustering, which identifies groups of subjects with similar properties and stores them in the same table, reducing the number of columns and thus the number of `NULL` entries.[9]

Property tables are best suited to RDF data with a fixed schema. If schema are present to define a limited subset of predicates relevant to a given class of subjects, each class can be stored in its own table with fewer columns, reducing (but not eliminating) the issue of `NULL` values. Because the type of an object for a given predicate is often specified for the schema, this facilitates efficient compression of literal predicates, which exist in a column of homogeneous type.[2]

In practice, the biggest shortcoming of property tables is that they lack an elegant solution for handling the case where a subject has multiple values for a given predicate, which is common in knowledge graphs. Additionally, when subjects are split up across multiple property tables, the benefit of avoiding joins on a subject is offset by the added cost and complexity of performing unions across property tables.[10]

2.1.3 Vertical Partitioning

Vertical partitioning creates a new table for each predicate, with columns for the subject and object, and a row for each triple with that predicate. This benefits from the same efficiencies provided by schema as property tables, but has no artificial `NULL` values and can handle multiple values for a given predicate. The main drawback is that it requires a join for each predicate in a query, which

`rdf:type`

Subject	Object
:Alice	foaf:Person
:Bob	foaf:Person
_:p	foaf:Project

`foaf:age`

Subject	Object
:Alice	26
:Bob	21

Figure 3: An RDF database vertically partitioned by predicate. Taken from [2].

can be expensive for queries with many predicates.[11]

2.1.4 Other Approaches

There are other, less common approaches to storing RDF data than those listed above. Approaches focused more on the fact that RDF data describes a graph include *graph-based storage*, which often stores pointers directly in an effort to optimize *path queries* which require a traversal of the graph structure, and *tensor-based storage*, which uses a rank 3 tensor to store the 3 dimensional equivalent of an adjacency matrix, the third dimension annotating the predicate.[2]

2.2 Indexing of RDF Data

An RDF database is indexed to allow for efficient lookup of triples matching a given criteria in $O(1)$ or $O(\log(n))$ time, and if possible minimize the number and size of joins required to compute queries with . Most indices in practice are stored in B-trees or B+-trees, which allow for efficient lookup of values in worst case $O(\log(n))$ time, although some implementations use hash tables for faster lookup of values in a small range or radix trees when keys are expected to be short.[10]

2.2.1 Triple Indexes

Statement-based queries, which specify a subject, predicate, object, or some combination thereof, can be efficiently answered by a triple index, which is a data structure that allows for efficient lookup of triples matching a given pattern. If s, p, o denote values fixed in a query and $\mathbf{s}, \mathbf{p}, \mathbf{o}$ represent variables, there are $2^3 = 8$ possible query patterns: (s, p, o), $(\mathbf{s}, p, o)$, $(s, \mathbf{p}, o)$, $(s, p, \mathbf{o})$, $(\mathbf{s}, \mathbf{p}, o)$, $(\mathbf{s}, p, \mathbf{o})$, $(s, \mathbf{p}, \mathbf{o})$, and $(\mathbf{s}, \mathbf{p}, \mathbf{o})$, and indices can be created to optimize queries for each. [2]

RDF-3X[17], a popular RDF database for research purposes with a natively implemented triple table storage framework, constructs a distinct index for each permutation of s, p, and o. Indexes are also created for each ordered pair of s, p, and o and each of s, p, and o themselves, and these provide statistics such as the number of triples existing with a given subject or subject-object pair that allow the fastest index for a given query to be selected. Statement-based queries can then be accelerated by the use of the right index because it allows the results to be found with a single index lookup, and makes it possible to efficiently *merge join* the results with those from another statement if needed by presenting them in sorted order. The primary aim of this exhaustive indexing is to minimize the number and memory footprint of *self-joins* (joins where subsets of the same table are joined), which require a copying of data tens of times, which is often too large to fit in the main memory of a system.[17, 2]

2.3 SPARQL

There are many languages developed for querying RDF data ([3] describes 29 distinct RDF query languages), but the most popular is *SPARQL*[13], which is generally accepted as

the standard.[2]. Developed by W3C, SPARQL is a declarative language, meaning that it specifies what results should be returned, but not how they should be computed. There are four types of SPARQL query:

`SELECT`: A select query specifies a set of variables, and provides a set of triples that must be matched. The results are the set of bindings of the variables that satisfy the triples.

`CONSTRUCT`: A construct query returns a subgraph of the database matching a set of triples.

`ASK`: An ask query returns a boolean value indicating whether a match exists for a set of triples.

`DESCRIBE`: A describe query provides information about a resource in the database; the information provided is implementation dependent.

To give a sense for the basic structure of a SPARQL query, we consider the following very basic example, which finds every president of the United States, their birthplace, and their predecessor, if they have one:

```
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX dbr: <http://dbpedia.org/resource/>
PREFIX dbp: <http://dbpedia.org/property/>

SELECT ?president ?birthplace ?predecessor
WHERE {
  ?president a dbo:Person ;
             dbp:office "President of the United States"@en ;
             dbo:birthPlace ?birthplace .
  OPTIONAL {
    ?president dbp:predecessor ?predecessor .
    ?predecessor dbp:office "President of the United States"@en .
  }
} ORDER BY DESC(?predecessor)
```

The `PREFIX` definitions at the beginning of the query provide convenient prefixes for the URIs of the entities and predicates used in the query. Because URIs are generally URLs, they can be very verbose, and the prefix definition greatly simplifies the referencing of entities. Naturally, `dbo:birthPlace` is expanded to `http://dbpedia.org/ontology/birthPlace`, and likewise for all prefixes. The difference between predicates from the ontology and property prefixes is that ontological predicates exist as part of a resource's schema, in this case `dbo:Person`, while property predicates are miscellaneous properties of a resource, in this case `dbp:office`: all people have a birthplace(`dbo:birthPlace`), but not all people hold public office(`dbp:office`).

The `SELECT` statement specifies that the variables we want to return from the query are those named before `WHERE`. If we wanted to know whether there existed a president with a birthplace and predecessor, this would be an `ASK`, and so on for the other two query types.

president	birthplace	predecessor
http://dbpedia.org/resource/Millard_Fillmore	http://dbpedia.org/resource/Moravia,_New_York	http://dbpedia.org/resource/Zachary_Taylor
http://dbpedia.org/resource/Warren_G._Harding	http://dbpedia.org/resource/Blooming_Grove,_Ohio	http://dbpedia.org/resource/Woodrow_Wilson
http://dbpedia.org/resource/Theodore_Roosevelt	http://dbpedia.org/resource/New_York_City	http://dbpedia.org/resource/William_McKinley
http://dbpedia.org/resource/Woodrow_Wilson	http://dbpedia.org/resource/Staunton,_Virginia	http://dbpedia.org/resource/William_Howard_Taft
http://dbpedia.org/resource/Calvin_Coolidge	http://dbpedia.org/resource/Plymouth_Notch,_Vermont	http://dbpedia.org/resource/Warren_G._Harding
http://dbpedia.org/resource/Rutherford_B._Hayes	http://dbpedia.org/resource/Delaware,_Ohio	http://dbpedia.org/resource/Ulysses_S._Grant
http://dbpedia.org/resource/James_Madison	http://dbpedia.org/resource/Port_Conway,_Virginia	http://dbpedia.org/resource/Thomas_Jefferson
http://dbpedia.org/resource/James_Madison	http://dbpedia.org/resource/British_America	http://dbpedia.org/resource/Thomas_Jefferson
http://dbpedia.org/resource/James_Madison_as_Father_of_the_Constitution	http://dbpedia.org/resource/Port_Conway,_Virginia	http://dbpedia.org/resource/Thomas_Jefferson
http://dbpedia.org/resource/James_Madison_as_Father_of_the_Constitution	http://dbpedia.org/resource/British_America	http://dbpedia.org/resource/Thomas_Jefferson
http://dbpedia.org/resource/William_Howard_Taft	http://dbpedia.org/resource/United_States_Reports	http://dbpedia.org/resource/Theodore_Roosevelt
http://dbpedia.org/resource/William_Howard_Taft	http://dbpedia.org/resource/Cincinnati	http://dbpedia.org/resource/Theodore_Roosevelt
http://dbpedia.org/resource/William_Howard_Taft	http://dbpedia.org/resource/Ohio	http://dbpedia.org/resource/Theodore_Roosevelt

Figure 4: A sample of the results of the query. Screenshot from https://dbpedia.org/sparql/.

The `WHERE` clause contains the pattern to match against the graph: the triple statements that restrict the variables defined by a `?` or `$` prefix (all variables in this query use `?`, but the two prefixes are interchangeable). The first three lines of the pattern share a subject, the variable `?president`, and as such the first two are terminated with semicolons instead of periods, to indicate that the subject should be carried over to the next line. The first of these lines uses `a`, a reserved keyword that SPARQL uses to indicate that a resource belongs to a given schema or subclass thereof. The second line confirms the existence of a triple matching `(?president, dbp:office, "President of the United States"@en)`, and the third line establishes that `?birthplace` is any value for which there exists a triple of the form `(?president, dbo:birthPlace, ?birthplace)` for the `?president` in question. The `OPTIONAL` clause that follows specifies that if a given `?president` has a value of `?predecessor` for which the criteria specified in the clause are met, then the value of `?predecessor` should be returned as part of the output, but that a lack of such a value (as in the case of George Washington) should not prevent the query from returning a result for the `?president` in question.

The `ORDER BY` clause specifies that the results should be ordered by the value of the variable `?predecessor`, in descending order (per the `DESC` function). This is not strictly necessary, but because the implicit ordering of null results is the lowest possible, it allows us to see the results with a `?predecessor` value first.

You can run and modify this query yourself at this page. You may notice that it does not identify every president, and this is primarily due to inconsistencies in how Wikipedia and by extension DBpedia identifies that a person was once president of the United States.

A sample of the query's output can be seen in figure 4. Note that if there are multiple valid birthplace values for a given president (i.e. James Madison being born in both British America and Port Conway, Virginia), then those combinations will be returned as separate rows. This behavior can be prevented by the addition of a `DISTINCT` keyword before a variable one does not want to duplicate. The two different pages describing James Madison highlight a common issue with many large knowledge graphs, the existence of multiple resources referencing the same real-world entity[14].

3 Question Answering

The natural application of knowledge graphs such as Google's [20] representing information about relationships between real-world entities is in answering questions about those entities and their relationships. In the most common case, which represents more than 85% of queries in the *WebQuestions* dataset built on a sample of real-world queries [22, 5] so-called 'simple' queries are dependent on the existence of a single triple in the graph, and are reducible to a statement-based query. For example, the questions "Who is the president of Argentina?" and "What country is Alberto Fernández president of?" can be converted to finding the triples matching (?, `president_of`, `Argentina`) and (`Alberto_Fernandez`, `president_of`, ?) respectively, the logic of which can be represented in a single line `WHERE` clause.

3.1 Complex Query Types

Below is a classification of constraint types derived from [4]. Each represents a subset of problems a knowledge graph question-answering model should be able to solve, and thus a subset of the knowledge graph workload. These are not mutually exclusive, and a single query may involve multiple types of constraints. [4] For each code snippet provided below, assume that the prefixes from the earlier SPARQL example have been defined.

Multi-entity constraint A multi-entity constraint is when a query involves multiple entities, such as "What movies starring Uma Thurman were directed by Quentin Tarantino".

```
SELECT ?movie
WHERE {
  ?movie dbo:starring dbr:Uma_Thurman .
  ?movie dbo:director dbr:Quentin_Tarantino .
}
```

Type constraint A type constraint specifies the type of the answer, such as "What country is the Mona Lisa in", where "country" is explicitly specified as the answer type.

```
SELECT ?country
WHERE {
  ?country a dbo:Country .
  dbr:Mona_Lisa dbo:museum ?museum .
  ?museum dbo:country ?country .
}
```

Explicit temporal constraint An explicit temporal constraint specifies a time period, such as "What movies were released in 2019".

```
SELECT ?movie
WHERE {
  ?movie a dbo:Film .
  ?movie dbp:releaseDate ?date .
  FILTER (YEAR(?date) = 2019)
}
```

Implicit temporal constraint Implicit temporal constraints involve cues such as "the last olympics" or "since the signing of the Declaration of Independence". For an example we will demonstrate "What countries have been founded since the signing of the Declaration of Independence?".

```
SELECT ?country
WHERE {
  ?country a dbo:Country .
  ?country dbp:establishedDate ?date .
  dbr:United_States_Declaration_of_Independence dbp:dateRatified ?doiDate .
  FILTER (?date > ?doiDate)
}
```

Ordinal constraint Ordinal constraints require the answer to be ranked in some way, such as "What is the 3rd largest city in the US?".

```
SELECT ?city
WHERE {
  ?city a dbo:City .
  ?city dbo:country dbr:United_States .
  ?city dbo:populationTotal ?pop .
}
ORDER BY DESC(?pop)
LIMIT 3
```

Aggregation constraint Aggregation constraints involve summary statistics about a set of entities, such as "How many movies were released in 2019?".

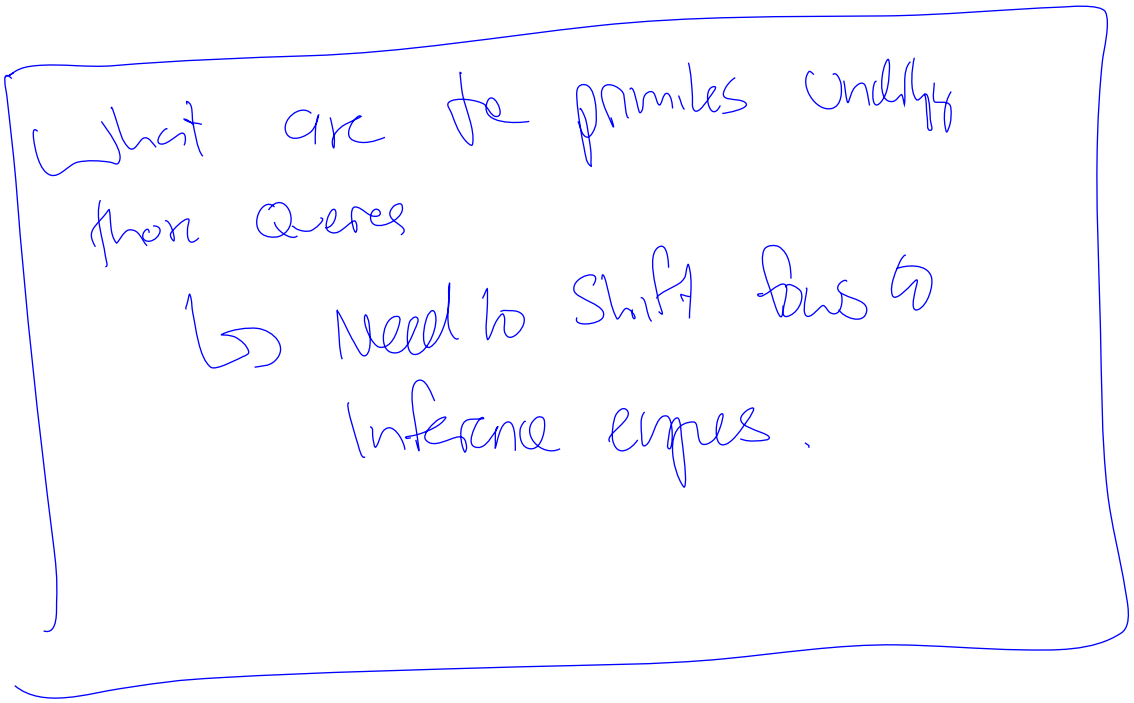
```
SELECT (COUNT(DISTINCT ?movie) AS ?count)
WHERE {
  ?movie a dbo:Film .
  ?movie dbp:releaseDate ?date .
  FILTER (YEAR(?date) = 2019)
```

```
}
```

These categories give some sense of the breadth of the knowledge graph workload, and the complexity of the queries that must be answered, which is generally not high on the scale of possible SPARQL queries, but requires efficient processing of basic triple patterns to enable high throughput question answering.

4 Further Work

There is much to be done in the domain of this paper. Further work could include a thorough discussion of algorithms for joins and relational algebra more broadly, which would better address the intricacies of where bottlenecks occur in queries on RDF databases. As part of such a discussion, it would be possible to then explore the benefits a massively parallel graph accelerator such as PIUMA[1] could provide to the query processing pipeline.



References

[1] Sriram Aananthakrishnan et al. "PIUMA: Programmable Integrated Unified Memory Architecture". In: *CoRR* abs/2010.06277 (2020). arXiv: 2010.06277. URL: https://arxiv.org/abs/2010.06277.

[2] Waqas Ali et al. *A Survey of RDF Stores & SPARQL Engines for Querying Knowledge Graphs*. 2021. DOI: 10.48550/ARXIV.2102.13027. URL: https://arxiv.org/abs/2102.13027.

[3] James Bailey et al. "Web and Semantic Web Query Languages: A Survey". In: *Proceedings of the First International Conference on Reasoning Web*. Msida, Malta: Springer-Verlag, 2005, pp. 35–133. ISBN: 3540278281. DOI: 10.1007/11526988_3. URL: https://doi.org/10.1007/11526988_3.

[4] Junwei Bao et al. "Constraint-Based Question Answering with Knowledge Graph". In: *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*. Osaka, Japan: The COLING 2016 Organizing Committee, Dec. 2016, pp. 2503–2514. URL: https://aclanthology.org/C16-1236.

[5] Jonathan Berant et al. "Semantic Parsing on Freebase from Question-Answer Pairs". In: *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*. Seattle, Washington, USA: Association for Computational Linguistics, Oct. 2013, pp. 1533–1544. URL: https://aclanthology.org/D13-1160.

[6] Tim Berners-Lee, James Hendler, and Ora Lassila. "The Semantic Web". In: *Scientific American* 284.5 (May 2001), pp. 34–43. URL: http://www.sciam.com/article.cfm?articleID=00048144-10D2-1C70-84A9809EC588EF21.

[7] Kurt Bollacker et al. "A Platform for Scalable, Collaborative, Structured Information Integration". In: *Sixth International Workshop on Information Integration on the Web* (July 2007). URL: https://www.aaai.org/Papers/Workshops/2007/WS-07-14/WS07-14-004.pdf.

[8] Kurt Bollacker et al. "Freebase: A Collaboratively Created Graph Database for Structuring Human Knowledge". In: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data* (June 2008). URL: https://dl.acm.org/doi/pdf/10.1145/1376616.1376746?casa_token=9_wcX5iS42sAAAAA:wfrW5cGlNdouQRqoOPu4j5Vf4lg4kih7UzcMIHiBkr3TJYf8kzyHK_g6g.

[9] Eugene Inseok Chong et al. "An efficient SQL-based RDF querying scheme". In: *Proceedings of the 31st international conference on Very large data bases*. 2005, pp. 1216–1227.

[10] "Chapter Five - Storage and Indexing of RDF Data". In: *RDF Database Systems*. Ed. by Olivier Curé and Guillaume Blin. Boston: Morgan Kaufmann, 2015, pp. 105–144. ISBN: 978-0-12-799957-9. DOI: https://doi.org/10.1016/B978-0-12-799957-9.00005-5. URL: https://www.sciencedirect.com/science/article/pii/B9780127999579000055.

[11] David Faye, Olivier Curé, and Guillaume Blin. "A Survey of RDF storage approaches". In: *Revue Africaine de la Recherche en Informatique et Mathématiques Appliquées* 15 (Feb. 2012), pp. 25. DOI: 10.46298/arima.1956.

[12] RDF Working Group. *RDF Schema 1.1*. W3C. 2014. URL: http://www.w3.org/TR/rdf-schema/ (visited on 02/25/2014).

[13] SPARQL Working Group. *SPARQL 1.1 Query Language*. W3C. 2013. URL: http://www.w3.org/TR/sparql11-query/.

[14] Aidan Hogan et al. "Knowledge Graphs". In: *CoRR* abs/2003.02320 (2020). arXiv: 2003.02320. URL: https://arxiv.org/abs/2003.02320.

[15] Graham Klyne and Jeremy J. Carroll. *Resource Description Framework (RDF): Concepts and Abstract Syntax*. W3C. 2004. URL: http://www.w3.org/TR/2004/REC-rdf-concepts-20040210/ (visited on 03/15/2015).

[16] Jens Lehmann et al. "DBpedia - A Large-scale, Multilingual Knowledge Base Extracted from Wikipedia". In: *Semantic Web Journal* 6 (Jan. 2014). DOI: 10.3233/SW-140134.

[17] Thomas Neumann and Gerhard Weikum. "RDF-3X: A RISC-Style Engine for RDF". In: *Proc. VLDB Endow.* 1.1 (Aug. 2008), pp. 647–659. ISSN: 2150-8097. DOI: 10.14778/1453856.1453927. URL: https://doi.org/10.14778/1453856.1453927.

[18] Thomas Pellissier Tanon, Gerhard Weikum, and Fabian Suchanek. "YAGO 4: A Reasonable Knowledge Base". In: *The Semantic Web*. Ed. by Andreas Harth et al. Cham: Springer International Publishing, 2020, pp. 583–596. ISBN: 978-3-030-49461-2.

[19] *Schema.org*. 2021. URL: https://schema.org/.

[20] Amit Singhal. "Introducing the Knowledge Graph: things, not strings". In: *Google Blog* (May 2012). URL: https://www.blog.google/products/search/introducing-knowledge-graph-things-not/.

[21] Denny Vrandečić and Markus Krötzsch. "Wikidata: A Free Collaborative Knowledgebase". In: *Commun. ACM* 57.10 (Sept. 2014), pp. 78–85. ISSN: 0001-0782. DOI: 10.1145/2629489. URL: https://doi.org/10.1145/2629489.

[22] Xuchen Yao. "Lean Question Answering over Freebase from Scratch". In: *Proceedings of the 2015 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*. Denver, Colorado: Association for Computational Linguistics, June 2015, pp. 66–70. DOI: 10.3115/v1/N15-3014. URL: https://aclanthology.org/N15-3014.